Lesson 3: Backpropagation

The chain rule read along a graph, gradients filled in by hand, then a backward() that does it for any expression.

Duration	About 1 h 40 min, plus homework.
File	`02_micrograd/engine.py`, started by pasting in the `Value` class from lesson 2, homework included. The name is micrograd's own, and lesson 4 adds `nn.py` next to it.
Before	Lesson 2 and its homework done, `tanh()` above all. Ideally the learner has watched the first 32 minutes of Karpathy's micrograd video, which replay lesson 2 with the same numbers. Not a blocker. No further: from 0:32 the video does block 1 of this lesson. If the lesson 2 `Value` still stores plain numbers as parents, or uses names other than `data`, `_prev` and `_op`, run the catch-up from the lesson 2 session review first.
Goals	The learner computes the gradient of every node of a small graph by hand with the chain rule, explains why the gradients must be filled from the output backwards, stores a function in an attribute, writes a `backward()` that works on any expression, and explains why `a + a` needs `+=`.

How to run this session

The rules from lessons 1 and 2 still hold.

- Ask for a prediction before every run.
- Give hints in order, only after a real attempt, and keep your hands off the keyboard.
- The learner reads errors aloud before anyone edits. Every message quoted here was checked on Python 3.14.
- Green boxes are for you.
- Purple boxes hold the target code of each step, also for you. Use them to see where the learner is heading and to pick the next hint, not to dictate. They build on the lesson 2 reference solution and end on the appendix of this guide.

The word "backpropagation" is allowed from the end of block 1, once the learner has done it by hand. The word "loss" waits for lesson 4.

Block	Time	Where
Warm-up and file setup	10 min	`engine.py`
1. Gradients by hand	35 min	`engine.py`
2. `_backward` and `backward()`	40 min	`engine.py`
3. The `a + a` bug	10 min	`engine.py`
4. Wrap-up and commit	5 min	`README.md`, terminal

Warm-up and file setup (10 min)

> **ASK**
> Without looking at the code: when Python runs `3 + Value(2.0)`, which methods run, and in what order?

EXPECTED (FOR YOU)

`int.__add__` returns `NotImplemented`, Python tries `Value.__radd__`, which wraps `3` in a `Value` and hands it to `__add__`. Block 2 stores functions on objects, so the learner must be at ease with "Python calls a method on the object" by now.

ASK

Predict `Value(0.8813735870195432).tanh()`.

EXPECTED (FOR YOU)

`Value(data=0.7071067811865476)`. This exact number comes back in block 2, step 4.

If `tanh()` is missing, write it now. The rest of the homework can wait, except `Value(2.0) + 3`: block 1 nudges a `Value` with `a + h`.

TASK

Create `02_micrograd/engine.py` with the `Value` class in the top `# %%` cell, pasted from `value.py`. Checks go in cells below, as before. If the lesson 2 stretch was skipped, `Value` has no `label` yet: add the parameter `label=""` to `__init__` now, because the graph, the printer and the neuron below all use it.

TARGET CODE (FOR YOU)

The `Value` class from the lesson 2 appendix, unchanged: `data`, `_prev`, `_op`, `label`, `__repr__`, `+` and `*` with plain numbers wrapped, `__radd__`, `__rmul__`, `tanh()`. Nothing new yet.

WATCH FOR

`AttributeError: 'Value' object has no attribute 'data'` on the first check: the class still uses its own attribute names. Rename now, before anything else is added.

Block 1: gradients by hand

Step 1: the graph from lesson 2, one node taller

TASK

Rebuild `a = 2`, `b = -3`, `c = 10`, `e = a * b`, `d = e + c` as `Value` objects, with labels. Then add `f = Value(-2.0)` and `L = d * f`. The learner predicts `L` first.

EXPECTED (FOR YOU)

`Value(data=-8.0)`. Call it `L` as the video does. In lesson 4 this node is the number the network tries to make small.

TARGET CODE (FOR YOU)

```
# %% The graph: L = (a * b + c) * f
a = Value(2.0, label="a")
b = Value(-3.0, label="b")
c = Value(10.0, label="c")
e = a * b
e.label = "e"
d = e + c
d.label = "d"
f = Value(-2.0, label="f")
L = d * f
L.label = "L"
print(L)  # Value(data=-8.0)
```

Step 2: the slopes, by nudging

ASK

Predict $\partial L/\partial a$. Then measure it as in lesson 2.

IF STUCK, IN THIS ORDER

1. "It is the trick from `value.py`: compute `L` once, then again with `a + h` in place of `a`."
2. "Write `L` as a function of `a` alone, with `b`, `c` and `f` frozen."

EXPECTED (FOR YOU)

With `h = 0.0001`, the slopes print as:

Nudge	Printed	Exact
a	6.000000000021544	6
b	-4.000000000008441	−4
c	-1.9999999999953388	−2
f	3.9999999999995595	4
d	-1.9999999999953388	−2
e	-1.9999999999953388	−2

Have the learner do `a`, `b` and `f` at least. Nudging `d` or `e` means recomputing only the part of the graph above them, which is a good question in itself: "What depends on `d`?"

TARGET CODE (FOR YOU)

```
# %% Slopes by nudging: rebuild L with one input moved by h
h = 0.0001
L_a = ((a + h) * b + c) * f
print((L_a.data - L.data) / h)  # 6.000000000021544
L_b = (a * (b + h) + c) * f
print((L_b.data - L.data) / h)  # -4.000000000008441
L_f = (a * b + c) * (f + h)
print((L_f.data - L.data) / h)  # 3.9999999999995595
L_d = (d + h) * f  # only what sits above d is recomputed
print((L_d.data - L.data) / h)  # -1.9999999999953388
```

`a + h` needs `Value + float`, homework item 1 of lesson 2. A function of plain floats, like lesson 2's `g(a, b, c)`, gives the same numbers and is just as good.

Step 3: the chain rule

ASK

Can you get $\partial L/\partial a = 6$ without nudging anything, from numbers you already know?

IF STUCK, IN THIS ORDER

1. "`L = d * f`. What is $\partial L/\partial d$?" Lesson 2 showed that the derivative of a product with respect to one factor is the other factor, so it is $f = -2$.
2. "`d = e + c`. If `e` moves by a tiny amount, how much does `d` move? And then how much does `L` move?" The same amount, then -2 times that amount.
3. "`e = a * b`. If `a` moves by `h`, `e` moves by `b * h`. Follow the nudge up to `L`."

EXPECTED (FOR YOU)

A nudge `h` on `a` becomes $-3h$ on `e`, stays $-3h$ on `d`, and becomes $(-2)(-3h) = 6h$ on `L`. So

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a} = (-2) \cdot (-3) = 6.$$

This is the chain rule from terminale, the derivative of a composed function, read along the graph: the gradient of a node is the gradient of the node above it, times the local derivative of the operation between them.

TASK

On paper, the gradient of all seven nodes. The learner fills a table, from `L` down.

EXPECTED (FOR YOU)

Node	$\partial L/\partial$ node	Why
`L`	1	Nudge `L` by `h`, `L` moves by `h`.
`f`	4	`L = d * f`, the other factor is `d = 4`.
`d`	–2	The other factor is `f = -2`.
`e`	–2	`d = e + c`, local derivative 1, times `d`'s –2.
`c`	–2	Same.
`a`	6	`e = a * b`, local derivative `b = -3`, times `e`'s –2.
`b`	–4	Local derivative `a = 2`, times –2.

The numbers match the nudges from step 2, without a single recomputation of `L`.

ASK

In which order did you fill the table? Could you have started with `a`?

EXPECTED (FOR YOU)

From `L` backwards. Each row needs the row of the node above it, so `a` cannot come first. Now say the word: walking the graph backwards from the output, multiplying by the local derivative at each step, is backpropagation. The whole of lesson 3 is making Python do this table.

Step 4: store it

TASK

Add an attribute `grad` to `Value`, `0.0` at creation, meaning "$\partial L/\partial$ this". Then, in a check cell, set all seven by hand from the table, using the `data` of other nodes rather than typed numbers: `d.grad = f.data`, and so on.

TARGET CODE (FOR YOU)

In `__init__`, one line: `self.grad = 0.0`. Then:

```
# %% Gradients by hand, from L down: (gradient above) x (local derivative)
L.grad = 1.0
f.grad = d.data * L.grad
d.grad = f.data * L.grad
e.grad = 1.0 * d.grad
c.grad = 1.0 * d.grad
a.grad = b.data * e.grad
b.grad = a.data * e.grad
```

The `* L.grad` and `1.0 *` change nothing here, but if the learner writes them, block 2 becomes a copy: each pair of lines is one `_backward`. Worth a nudge, not a requirement.

ASK

Why `L.grad = 1.0` and not `0.0`?

EXPECTED (FOR YOU)

It is $\partial L/\partial L$. The gradient of a node with respect to itself is 1, and it is the starting point every other gradient is multiplied from.

The lesson 2 stretch exercise printed the graph. If the learner did it, add `grad` to each line. Otherwise hand over this printer, because printing is not what this lesson teaches:

```
# %% A printer for the graph: one node per line, parents indented below
def show(v, depth=0):
    """Print v and everything it was computed from."""
    name = v.label or "?"
    op = f" ({v._op})" if v._op else ""
    print("  " * depth + f"{name} = {v.data:.4f}{op}  grad={v.grad:.4f}")
    for parent in v._prev:
        show(parent, depth + 1)
```

EXPECTED (FOR YOU)

`show(L)` prints, with the order of siblings possibly different:

```
L = -8.0000 (*)  grad=1.0000
  f = -2.0000  grad=4.0000
  d = 4.0000 (+)  grad=-2.0000
    e = -6.0000 (*)  grad=-2.0000
      a = 2.0000  grad=6.0000
      b = -3.0000  grad=-4.0000
    c = 10.0000  grad=-2.0000
```

WATCH FOR

`AttributeError: 'Value' object has no attribute 'grad'`. The objects were created before the class cell was re-run, lesson 1's recurring trap. Re-run the class cell, then rebuild the graph.

Step 5: one step, by hand (5 min)

ASK

You want `L` bigger. Which way do you nudge `a`? And `b`? What told you?

EXPECTED (FOR YOU)

`a` up, because its gradient is positive. `b` down, because its gradient is negative. The sign of the gradient points uphill.

TASK

For each of the four leaves, `leaf.data += 0.01 * leaf.grad`. Rebuild `e`, `d` and `L` from the nudged leaves and predict the new `L`.

EXPECTED (FOR YOU)

`-7.286496`, up from `-8.0`. Every leaf moved uphill at once, and `L` followed. Lesson 4 does the same with a minus sign, to push a number down instead. Chapter 2 of 3Blue1Brown called that gradient descent.

TARGET CODE (FOR YOU)

```
# %% One step uphill: every leaf moves along its gradient
a.data += 0.01 * a.grad
b.data += 0.01 * b.grad
c.data += 0.01 * c.grad
f.data += 0.01 * f.grad
e = a * b
d = e + c
L = d * f
print(L)  # Value(data=-7.286496)
```

The leaves are now changed. Block 2 starts by re-running the graph cell of step 1.

Block 2: `_backward` and `backward()`

Step 1: each operation knows its own rule

ASK

For `d = e + c` you wrote `e.grad = d.grad` and `c.grad = d.grad`. For `e = a * b`, `a.grad = b.data * e.grad` and `b.grad = a.data * e.grad`. Where in the code does Python know that `d` was made by `+`?

EXPECTED (FOR YOU)

In `__add__`, at the moment `out` is built. That is the one place where `self`, `other` and `out` are all at hand. The plan: each `__add__` and `__mul__` prepares the function that will later push `out.grad` to the two parents, and stores it on `out`. Nothing runs yet; the gradients are not known until `L` exists.

Functions as values are new. Take five minutes on unrelated code before the task:

```
# %% Functions are objects, and a nested function remembers its surroundings
import math

root = math.sqrt   # no parentheses: the function itself, not a call
print(root(16.0))  # 4.0


def make_adder(k):
    """Build and return a function that adds k."""

    def add(x):
        return x + k   # add remembers the k it was built with

    return add


add3 = make_adder(3)
print(add3(10))    # 13
```

ASK

`add3` was built inside `make_adder`, which finished long ago. How does `add3` still know `k`?

EXPECTED (FOR YOU)

A function defined inside another keeps the variables that were around it. The same mechanism lets a function defined inside `__add__` keep `self`, `other` and `out` after `__add__` has returned.

TASK

In `__add__`, define a function `_backward` that updates `self.grad` and `other.grad` from `out.grad`, and store it as `out._backward`. Same for `__mul__`. A leaf has no parents, so its `_backward` does nothing. Then, in a cell, rebuild the graph and run the three functions by hand, in the order of block 1:

```
# %% Checks: push the gradients down by hand
L.grad = 1.0
L._backward()
d._backward()
e._backward()
show(L)  # the same seven gradients as block 1
```

IF STUCK, IN THIS ORDER

1. "Inside `__add__`, which three objects does the rule need? Are they all in scope?"
2. "What does a leaf's `_backward` have to do? `lambda: None` is a function that does nothing, and it can be set in `__init__`."
3. Write `_backward` for `+` first, run the check with only `L` and `d`, then do `*`.

WATCH FOR

- `TypeError: 'NoneType' object is not callable`. `out._backward = _backward()` with parentheses, so Python called the function and stored what it returned. The message says what was called: `None`.
- `AttributeError: 'Value' object has no attribute '_backward'`. `out._backward = _backward` is missing, and `__init__` sets no default either.
- Every gradient stays `0.0`, `L`'s included. `L.grad = 1.0` was forgotten. Ask: "Who sets `L.grad`? Nobody above it can."
- Every gradient stays `0.0` except `L`'s. `out._backward = _backward` is missing, and the do-nothing default from `__init__` ran instead.
- `self.grad = out.grad` with `=` works for now. Leave it. Block 3 is about that.

TARGET CODE (FOR YOU)

```
    def __init__(self, data, _children=(), _op="", label=""):
        # data, grad, _prev, _op, label: unchanged
        self._backward = lambda: None  # a leaf has nothing to push down

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")

        def _backward():
            self.grad = out.grad  # local derivative 1; += in block 3
            other.grad = out.grad

        out._backward = _backward  # stored, not called
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")

        def _backward():
            self.grad = other.data * out.grad  # local derivative: the other factor
            other.grad = self.data * out.grad

        out._backward = _backward
        return out
```

`=` on purpose: block 3 finds why it must be `+=`. If the learner writes `+=` straight away, let it be and ask why in block 3.

Step 2: the order matters

ASK

Rebuild the graph so every `grad` is back to `0.0`, set `L.grad = 1.0`, then call the three functions in the order the nodes were created, `e`, `d`, `L`. Predict `show(L)` first.

EXPECTED (FOR YOU)

`a`, `b`, `c` and `e` get `0.0`, while `d` gets `-2.0` and `f` gets `4.0`. `e._backward` ran while `e.grad` was still `0.0` and pushed that zero to `a` and `b`; `d._backward` did the same to `e` and `c`. A node's `_backward` may only run once its own `grad` is final, so after every node that depends on it. Without the rebuild, the gradients left over from step 1 make the wrong order print the right numbers.

TARGET CODE (FOR YOU)

```
# %% Wrong order: the order of creation (re-run the graph cell first)
L.grad = 1.0
e._backward()
d._backward()
L._backward()
show(L)  # a, b, c, e at 0; d at −2, f at 4
```

WATCH FOR

a prints `grad=-0.0000`. With =, `a.grad` is `b.data * e.grad`, that is `-3.0 * 0.0`, and a float keeps the sign of a zero. It is equal to `0.0`. With += it would print `0.0000`, because `0.0 + -0.0` is `0.0`.

Step 3: `backward()`

TASK

A method `backward()` on `Value`, so that `L.backward()` sets `L.grad` to 1 and calls every `_backward` in a valid order, for any graph. The only information it has is each node's `_prev`. Check with `show(L)` on the block 1 graph.

IF STUCK, IN THIS ORDER

1. "The printer visits every node under `L`, parents after their child. Build a list the same way, but append a node only after visiting all its parents. In that list, where does `L` end up?" Last, so the list is walked in reverse.
2. "A node reachable by two paths must appear once. Keep a set of nodes already seen." The lesson 2 printer prints such a node twice, which is harmless for printing but not here.
3. If recursion is the obstacle, write the printer from lesson 2 first, then adapt it.

EXPECTED (FOR YOU)

A nested function that visits a node, recurses into its parents, then appends the node to a list, plus a `seen` set. Then `self.grad = 1.0` and a loop over the list in reverse. The video calls that list a topological order. The word is worth knowing but not the point.

TARGET CODE (FOR YOU)

```
    def backward(self):
        """Fill grad on every node under self, from self back to the leaves."""
        order = []
        seen = set()

        def visit(v):
            """Append v to order after everything it was computed from."""
            if v not in seen:
                seen.add(v)
                for parent in v._prev:
                    visit(parent)
                order.append(v)  # after all its parents

        visit(self)
        self.grad = 1.0
        for v in reversed(order):  # self first, leaves last
            v._backward()
```

Then re-run the graph cell, `L.backward()` and `show(L)`: the seven gradients of block 1. A version where `visit` is a separate function taking `order` and `seen` as arguments is just as right.

WATCH FOR

- `TypeError: cannot use 'Value' as a set element (unhashable type: 'Value')`. A `__eq__` was added to `Value`, following the lesson 1 detour, and Python then refuses to put the object in a set. Remove `__eq__`. Two different nodes with the same `data` must stay different nodes.
- `TypeError: 'NoneType' object is not callable` or `AttributeError: 'Value' object has no attribute '_backward'`, on a leaf. `backward()` is the first code to call a leaf's `_backward`, and `__init__` sets it to `None` or not at all instead of the do-nothing default.
- `L.backward()` twice in a row, once `+=` is in, does not give the same numbers again. `L.grad` is reset to 1, but every other gradient keeps growing: `d` and `f` double, `e` and `c` triple, `a` and `b` quadruple. Ask whether that is a bug. `backward()` assumes every gradient starts at `0.0`, and lesson 4 resets them before each pass.

Step 4: a neuron

The derivative of `tanh` is needed first.

ASK

On paper, with $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$: what is $\tanh'(x)$?

EXPECTED (FOR YOU)

The quotient rule gives $4e^{2x}/(e^{2x} + 1)^2$, and expanding $1 - \tanh(x)^2$ gives the same fraction. So $\tanh'(x) = 1 - \tanh(x)^2$, which is handy: the derivative comes from the output, `1 - out.data**2`, with no `exp` to recompute. If the algebra drags, accept the formula and check it by nudging `tanh` at `0.8813735870195432` with `h = 0.0001`: the nudge gives `0.4999646454939555`, and `1 - 0.7071067811865476**2` gives `0.4999999999999999`.

TARGET CODE (FOR YOU)

```
    def tanh(self):
        e = math.exp(2 * self.data)
        t = (e - 1) / (e + 1)
        out = Value(t, (self,), "tanh")

        def _backward():
            self.grad = (1 - t**2) * out.grad  # from the output t, not self.data

        out._backward = _backward
        return out
```

```
# %% Check tanh' by nudging, before trusting the rule
h = 0.0001
x = 0.8813735870195432
print((Value(x + h).tanh().data - Value(x).tanh().data) / h)  # 0.4999646454939555
print(1 - Value(x).tanh().data ** 2)  # 0.4999999999999999
```

TASK

`_backward` for `tanh()`. Then a neuron with two inputs, the video's numbers so the learner can check against it later:

```
# %% A neuron: two inputs, two weights, a bias, tanh
x1 = Value(2.0, label="x1")
x2 = Value(0.0, label="x2")
w1 = Value(-3.0, label="w1")
w2 = Value(1.0, label="w2")
b = Value(6.8813735870195432, label="b")
n = x1 * w1 + x2 * w2 + b
n.label = "n"
o = n.tanh()
o.label = "o"
```

ASK

Predict `n` and `o`, then run `o.backward()` and `show(o)`.

EXPECTED (FOR YOU)

`n` is `0.8813735870195432`, the warm-up number, and `o` is `0.7071067811865476`. The gradients:

o	n	b	x1*w1	x2*w2	x1	w1	x2	w2
1	0.5	0.5	0.5	0.5	−1.5	1.0	0.5	0.0

Printed values carry float noise, such as `-1.4999999999999996` for `x1`.

ASK

`w2.grad` is 0. Does that mean `w2` does not matter?

EXPECTED (FOR YOU)

It means `w2` does not matter for this input, because `x2 = 0` and `w2` is multiplied by it. A weight only gets a gradient from inputs that are not zero. Follow-up: `x1.grad` is -1.5, but `x1` is an input, not a weight. In lesson 4 only weights and biases get nudged; the inputs are data. The gradient on `x1` is computed anyway and does no harm.

WATCH FOR

`n.grad` prints `0.2232` instead of `0.5`. The `tanh` rule used `1 - self.data**2`, the input, instead of `1 - out.data**2`, the output. The nudge check from the start of this step catches it.

Block 3: the `a + a` bug (10 min)

ASK

`a = Value(3.0)`, `b = a + a`, `b.backward()`. Predict `a.grad` on paper, then run.

EXPECTED (FOR YOU)

On paper, `b` is `2 * a`, so the gradient is 2. With = in `_backward`, the code prints `1.0`.

IF STUCK, IN THIS ORDER

1. "`a` is both parents of `b`. How many times does `b._backward` write to `a.grad`?"
2. "What does the second write do to the first?"

EXPECTED (FOR YOU)

Both writes store `1.0`, and the second one erases the first. The fix is += in every `_backward`, which is why `grad` starts at `0.0` and not at nothing. Any node used twice in a graph is hit by this, and a network reuses every input many times.

TARGET CODE (FOR YOU)

= becomes += on the five lines that write a gradient: two in `__add__`, two in `__mul__`, one in `tanh`. For example `self.grad += other.data * out.grad`. Then:

```
# %% A node used twice
a = Value(3.0)
b = a + a
b.backward()
print(a.grad)  # 1.0 with =, 2.0 with +=
```

TASK

With += everywhere: `a = Value(-2.0)`, `b = Value(3.0)`, `f = (a * b) * (a + b)`. The learner predicts `f`, then $\partial f/\partial a$ on paper, then runs `f.backward()`.

EXPECTED (FOR YOU)

`f` is `-6.0`. On paper, the product rule gives $\partial f/\partial a = b(a+b) + ab = 3 - 6 = -3$ and $\partial f/\partial b = a(a+b) + ab = -2 - 6 = -8$. The code prints `-3.0` and `-8.0`. With = instead of += it would have printed something else, and the learner can say what by following the writes.

TARGET CODE (FOR YOU)

```
# %% Product rule, checked by backward()
a = Value(-2.0)
b = Value(3.0)
f = (a * b) * (a + b)
f.backward()
print(f, a.grad, b.grad)  # Value(data=-6.0) -3.0 -8.0
```

With =, the nodes `a * b` and `a + b` each write `a.grad` and `b.grad` once, and the last writer wins: `-6.0 -6.0` if `a + b` runs last, `3.0 -2.0` if `a * b` does. With `_prev` as a set, which one runs last is not fixed, so the learner may see either.

Block 4: wrap-up and commit

ASK

In one sentence: what does `L.backward()` do?

EXPECTED (FOR YOU)

It fills $\partial L/\partial$ node for every node, walking from `L` back to the leaves, each node multiplying the gradient it received by its own local derivative. Nothing in it needed a formula for `L` as a whole, which is why it works for a graph of any size.

```
git add 02_micrograd/engine.py
git commit -m "micrograd: backpropagation by hand, then backward() with the chain rule"
git push
```

Then a few sentences in the `README.md` journal, committed too.

Homework

1. Watch the micrograd video from 0:32 to 1:39. Up to 1:27 it replays today's lesson with the same numbers; from 1:27 it does the homework items below. Listen for the English words: gradient, backward pass, chain rule, topological sort, leaf node.
2. `_backward` for `__pow__`, using $\left(x^k\right)' = kx^{k-1}$. `Value(3.0) ** 2` gives `9.0` and a gradient of `6.0` on the input.
3. `exp()`, with `math.exp`. Its derivative is itself, so the rule reads from `out.data`. `Value(0.0).exp()` gives `1.0` with gradient `1.0`; `Value(1.0).exp()` gives `2.718281828459045`, and its gradient is that same number.
4. `__truediv__` as `self * other ** -1`, and `__neg__` as `self * -1`. No new rule is needed: `*` and `**` already know theirs. `Value(6.0) / Value(3.0)` gives `2.0`, with gradients `0.3333333333333333` on the 6 and `-0.6666666666666666` on the 3.
5. The video's exercise at 1:27: rebuild the neuron with `tanh` spelled out as `(e - 1) / (e + 1)` where `e = (2 * n).exp()`. `o.backward()` must give the same nine gradients as block 2, step 4. The graph is longer, the gradients are the same.

Next: lesson 4 turns `Value` into a neuron class, a layer and a small network, writes the number to make small, and trains on four examples. That is the video from 1:44.

Appendix: a reference solution

Show this only after the lesson, next to the learner's own file, and go through the differences together. It covers what the lesson asks for; the lesson 2 homework operations are left out.

```
# %% The class
import math


class Value:
    """A number that remembers how it was computed, and its gradient."""

    def __init__(self, data, _children=(), _op="", label=""):
        """Called by Value(2.0): a leaf unless _children names its parents."""
        self.data = data
        self.grad = 0.0               # dL/dthis, filled by backward()
        self._backward = lambda: None  # a leaf has nothing to push down
        self._prev = set(_children)
        self._op = _op
        self.label = label

    def __repr__(self):
        """What print shows."""
        return f"Value(data={self.data})"

    def __add__(self, other):
        """self + other, plus the rule that pushes out.grad back to both."""
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")

        def _backward():
            """Local derivative of + is 1 for both parents."""
            self.grad += out.grad
            other.grad += out.grad

        out._backward = _backward
        return out

    def __mul__(self, other):
        """self * other, same pattern as +."""
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")

        def _backward():
            """Local derivative of * is the other factor."""
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad

        out._backward = _backward
        return out
```

The class continues:

```
    def tanh(self):
        """tanh of self, from math.exp, with one parent."""
        t = (math.exp(2 * self.data) - 1) / (math.exp(2 * self.data) + 1)
        out = Value(t, (self,), "tanh")

        def _backward():
            """tanh' = 1 - tanh^2, read from the output."""
            self.grad += (1 - t**2) * out.grad

        out._backward = _backward
        return out

    def backward(self):
        """Fill grad on every node under self, from self back to the leaves."""
        order = []
        seen = set()

        def visit(v):
            """Append v to order after everything it was computed from."""
            if v not in seen:
                seen.add(v)
                for parent in v._prev:
                    visit(parent)
                order.append(v)  # after all its parents

        visit(self)
        self.grad = 1.0
        for v in reversed(order):
            v._backward()
```

Points to compare

ASK

In `tanh`, the rule uses `t` and not `out.data`. Are they the same thing? Which one would you write?

EXPECTED (FOR YOU)

They are the same number, since `out.data` was set to `t`. Both are fine. The point to notice is that neither is `self.data`: the derivative of `tanh` is cheapest to compute from its output.

ASK

Why does `_backward` live on `out`, and not on `self`?

EXPECTED (FOR YOU)

The function pushes `out.grad` down to its parents, so it belongs to the node that was produced. `self` may be a parent of many nodes, each with its own rule, and `self._backward` is already taken by the operation that produced `self`.

ASK

`backward()` sets `self.grad = 1.0` after building the order, not before. Would it matter?

EXPECTED (FOR YOU)

No. Building the order reads only `_prev` and touches no gradient, so the line can go before or after it. What matters on a second call is = against +=: `self.grad = 1.0` resets the top node, while every `_backward` keeps adding to what is already there. Lesson 4 adds a step that resets every gradient to zero before each `backward()`.